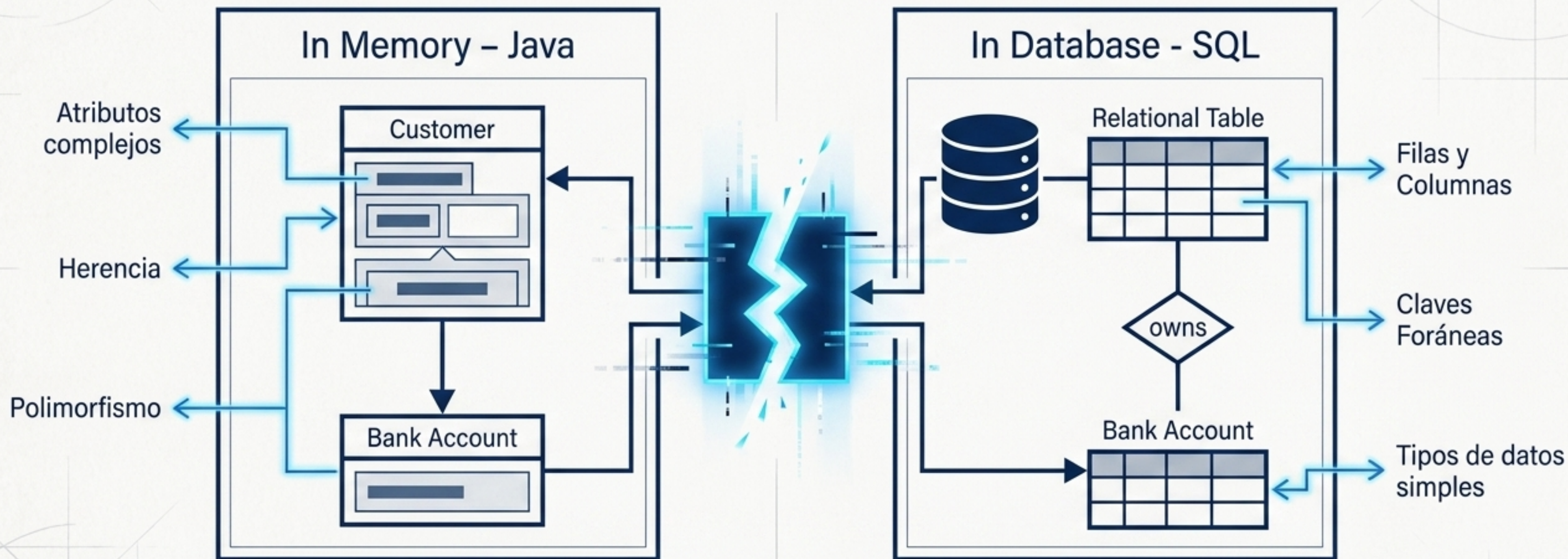


Fuente: Tema 19

Arquitectura Limpia en Bases de Datos Objeto-Relacionales

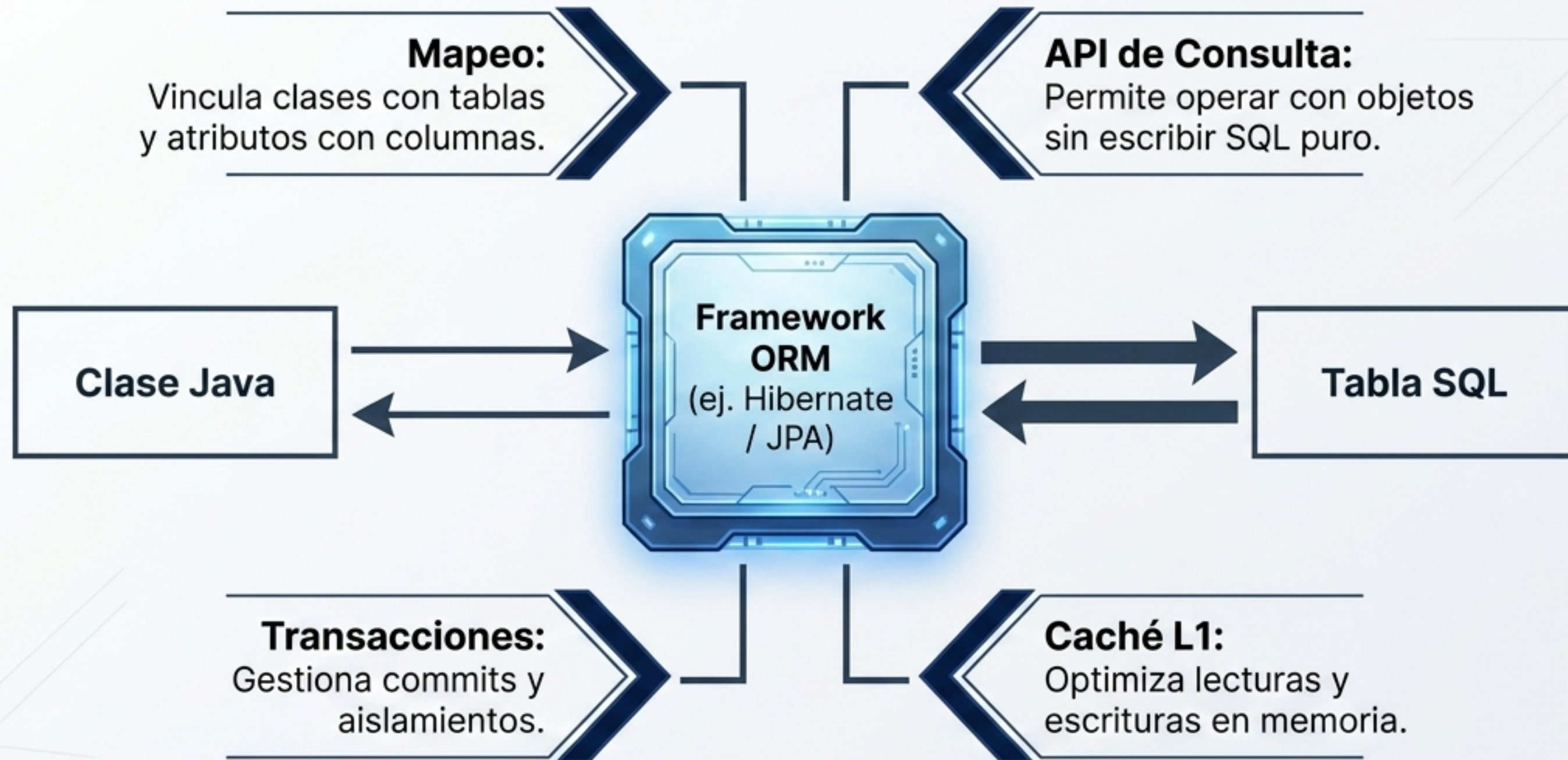
De la abstracción de objetos a la interfaz gráfica:
un viaje seguro y estructurado a través de Java.

El Desajuste Objeto-Relacional: Dos lenguajes incompatibles



Los lenguajes orientados a objetos y las bases de datos relacionales requieren una capa de traducción nativa para interactuar sin fricción.

ORM: El puente traductor automatizado



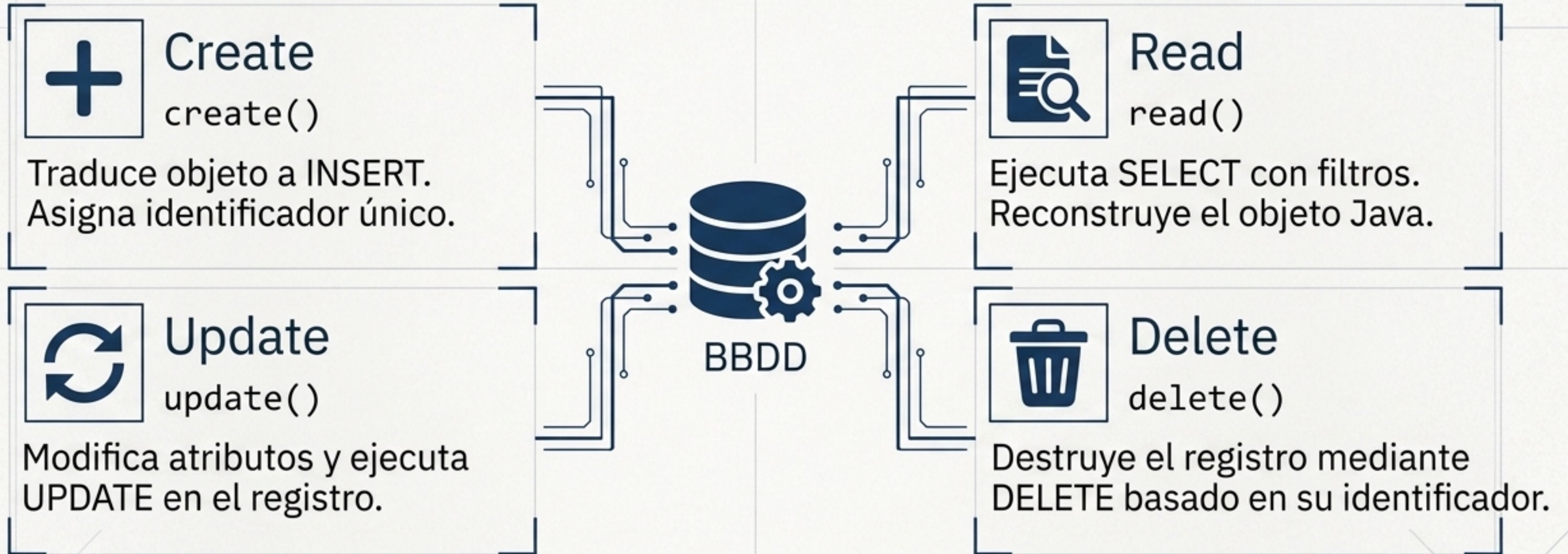
El Object-Relational Mapping (ORM) proporciona una capa de abstracción que permite manipular datos relacionales de forma transparente mediante objetos de Java.

Matriz de Decisión: ORM (Hibernate) vs. SQL Nativo

	Framework ORM (Hibernate)	Implementación Directa (SQL Nativo)
Complejidad de Relaciones	Alta (ideal para redes de objetos interconectados, ej: Profesores -> Estudiantes -> Asignaturas).	Baja (ideal para entidades aisladas, ej: Solo Estudiantes).
Mantenibilidad de Código	Alta abstracción, código más limpio.	Acoplamiento directo a la sintaxis del motor de base de datos.
Control de Rendimiento	Delegado al framework.	Control absoluto y optimización manual de consultas.

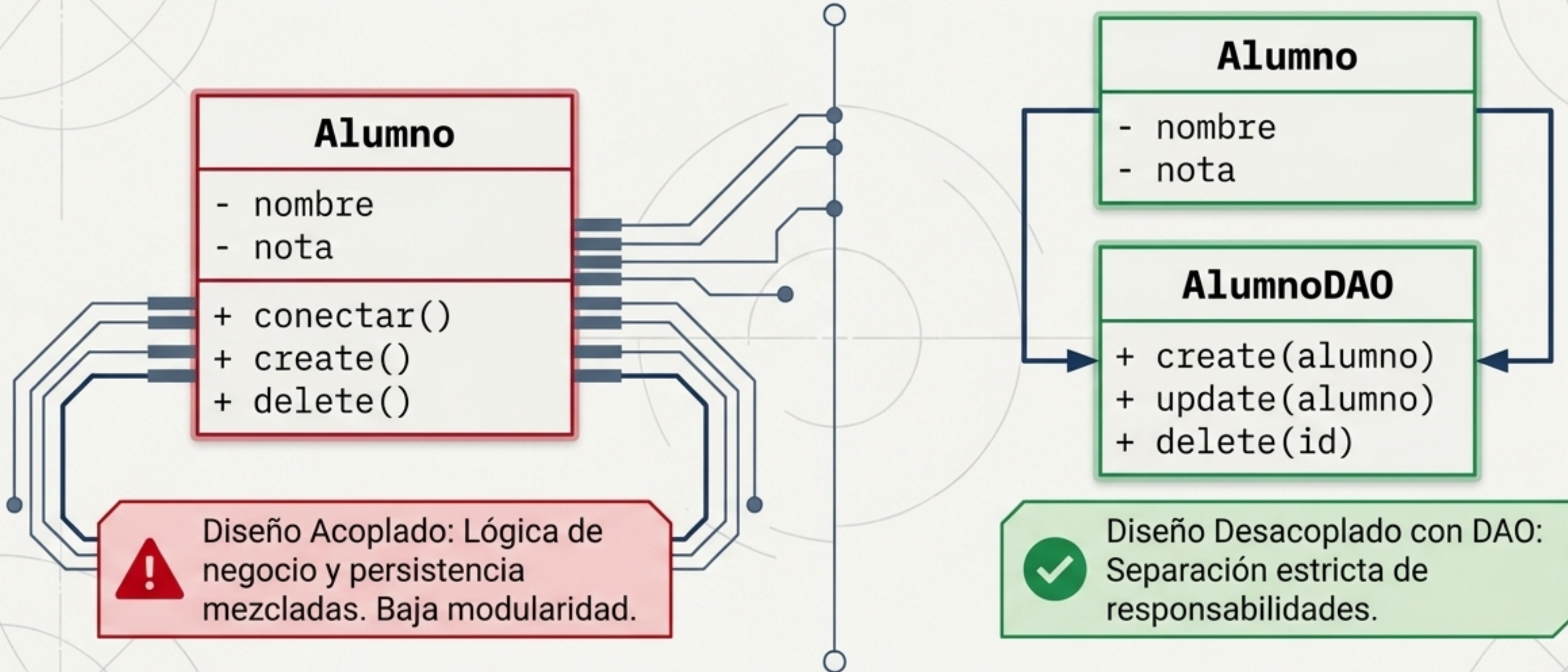
Veredicto del Caso Práctico: Para un sistema que gestiona estudiantes aislados sin relación jerárquica compleja, una implementación directa con SQL nativo resulta más eficiente y directa.

Operaciones Base: El estándar CRUD



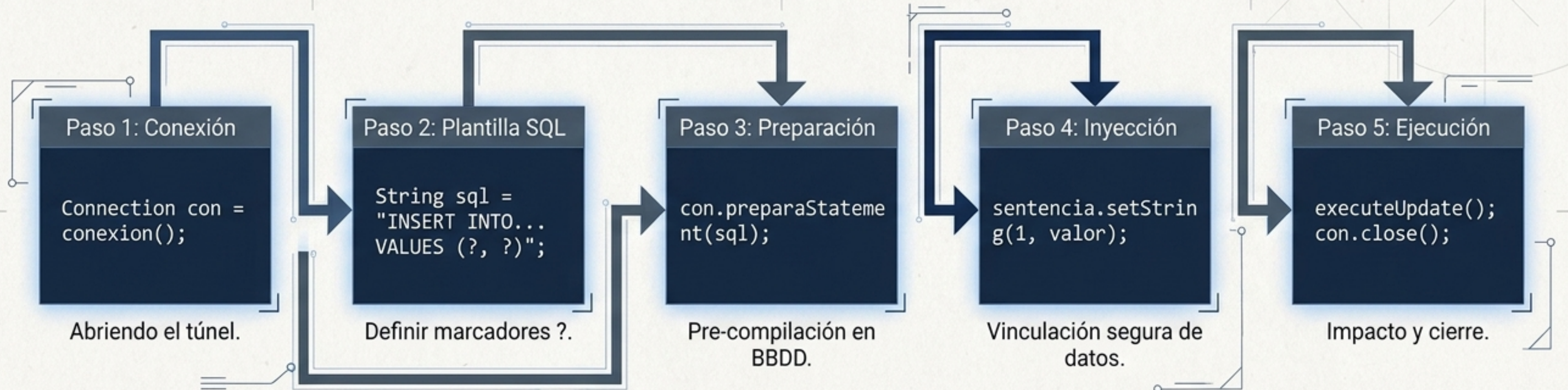
Toda interacción con la base de datos, por compleja que sea, se descompone en estas cuatro operaciones atómicas.

Arquitectura Limpia: El patrón DAO (Data Access Object)



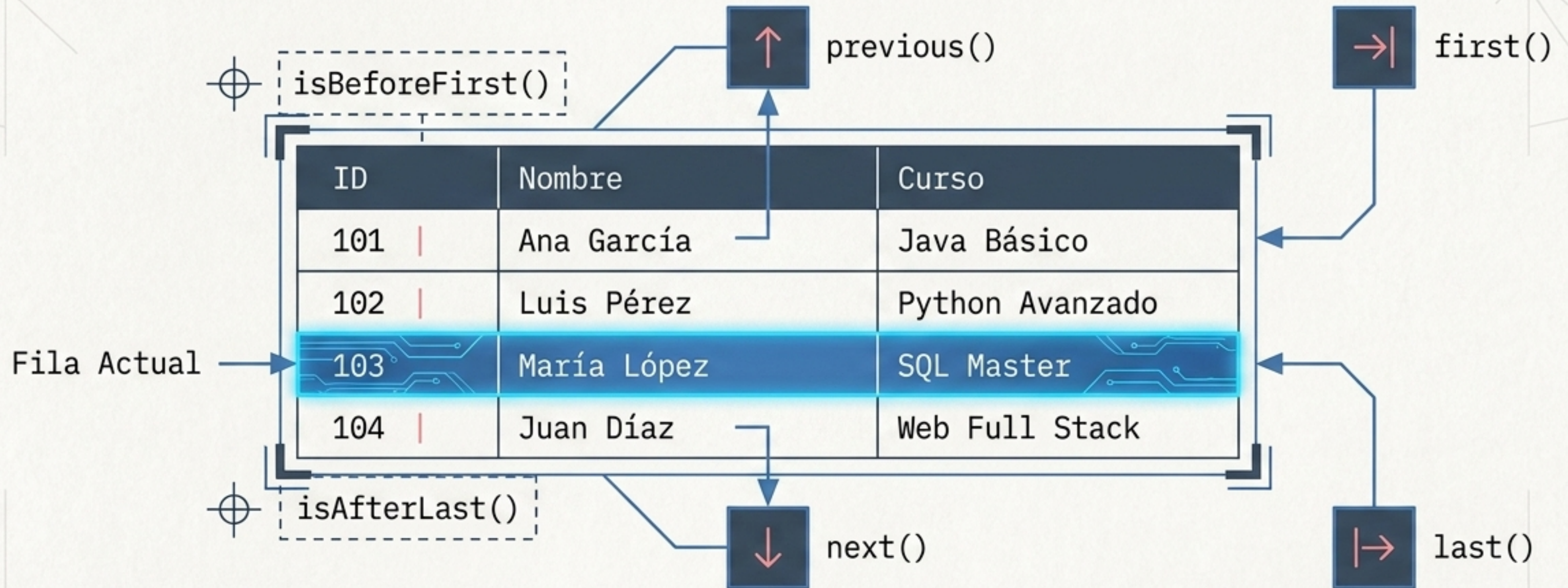
El DAO aísla el acceso a los datos. Si cambiamos de MySQL a Oracle, la clase Alumno permanece intacta.

Flujo de Ejecución: Anatomía de una operación DAO



Independientemente de la operación CRUD, el ciclo de vida de la sentencia parametrizada en Java mantiene un flujo riguroso para garantizar estabilidad.

Navegación de Resultados: El **Cursor ResultSet**



El **ResultSet** no es un array estático, sino un flujo dinámico. El cursor debe desplazarse explícitamente para leer y mapear cada registro hacia objetos Java.

La Amenaza a la Arquitectura: Inyección SQL



' ; DROP TABLE Products; --

```
String consulta = "SELECT * FROM usuarios WHERE nombre = '" + input + "'";
```

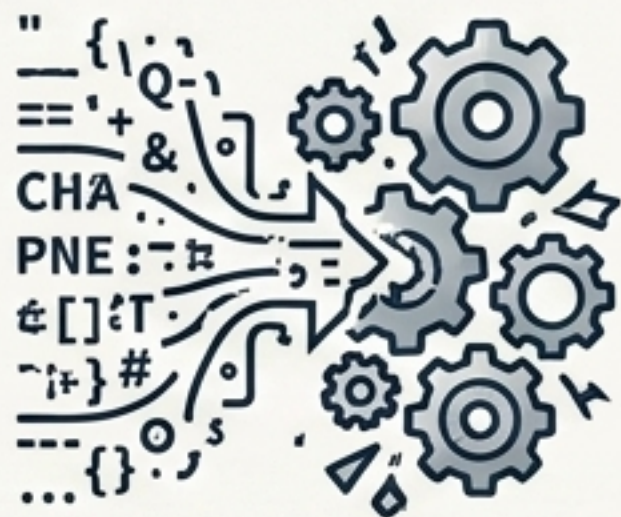
```
SELECT * FROM Products WHERE name = ' '; DROP TABLE Products; --
```

Al usar consultas dinámicas tradicionales, los datos de usuario se convierten inadvertidamente en comandos ejecutables.

El Escudo de Defensa: Sentencias Parametrizadas

Concatenación Dinámica

```
"... WHERE nombre = '" + input + "'"
```



El motor fusiona sintaxis y datos.
El input malicioso altera la estructura.

PreparedStatement

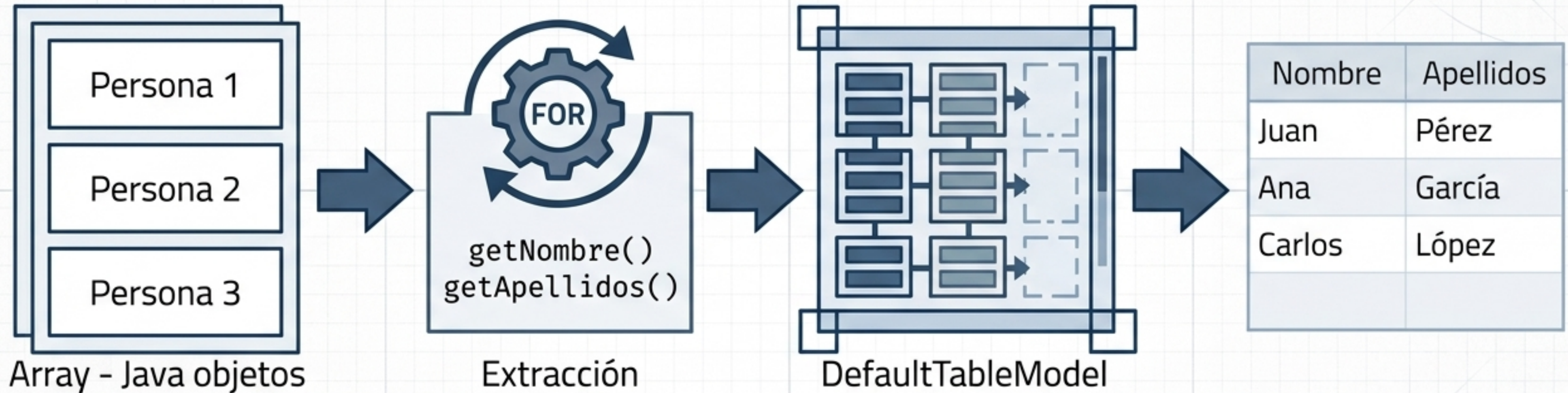
```
"... WHERE nombre = ?"
```



Separación estricta: la BBDD pre-compila la estructura primero. Los datos ingresan como literales inertes y nunca alteran la sintaxis.

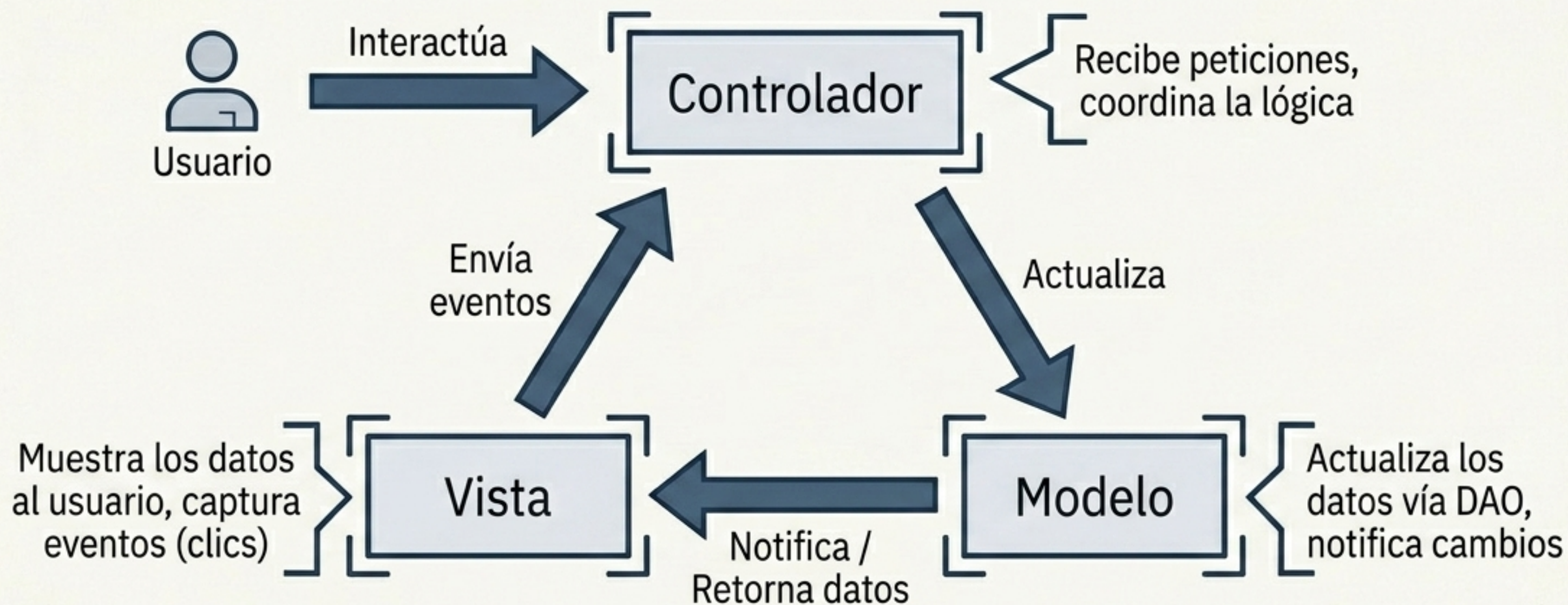
La parametrización anula la inyección SQL mitigando el riesgo desde el motor de la base de datos, no solo desde la aplicación.

Interfaz Gráfica: Del Objeto a la Vista



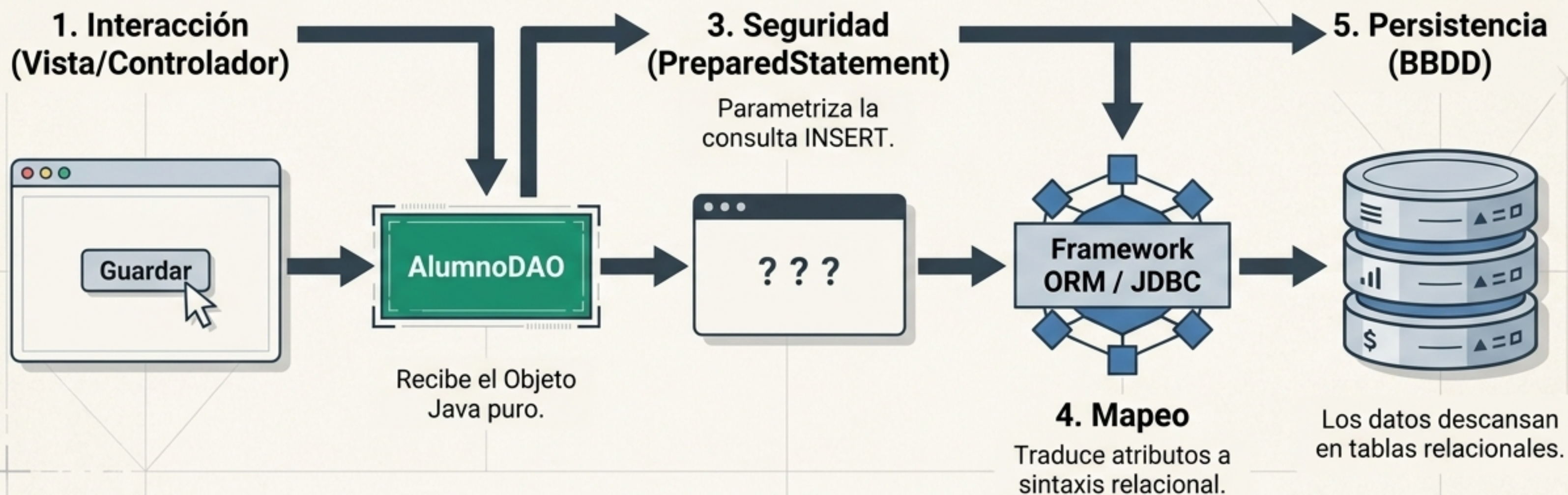
Construir una interfaz moderna requiere desempaquetar las listas de objetos y estructurarlas secuencialmente en los modelos de las tablas gráficas.

Arquitectura de Interfaz: El patrón MVC



MVC promueve la máxima escalabilidad al evitar que la pantalla gráfica interactúe directamente con la base de datos.

Síntesis Arquitectónica: El Sistema Integrado



Un sistema robusto en Java no se define solo por cómo guarda datos, sino por cómo estructura el viaje de esa información: desacoplado (DAO), seguro (Parametrización), organizado (MVC) y abstraído (ORM).